PROOFLINE

Agent Outcome Assurance: verify what your AI agent actually did

Master Blueprint v2.1 (phase-wise)
Proofline vision + 12-week Enterprise AI Reliability plan + Krish FDE roadmap, merged into one executable plan

Prepared 24 Oct 2026 | Phase-wise plan: 6 phases, exit-test driven

Is document me kya hai

- Senior architect analysis: Proofline doc ki 14 kamiyan aur faisle, market wedge, asli agent problems.
- Final product: Proofline Lab (white-box) + Proofline Scout (permission-gated black-box audit agent).
- Architecture, ADRs, data model, verifier spec, metrics (pass^k), replay levels, failure taxonomy.
- End-to-end workflows, real-world testing tiers, outreach playbook aur email templates.
- Deployment, security, CI/CD, runbook; testing strategy (Proofline Bench); phase-wise plan, risks, cut ladder.
- Internship conversion: portfolio, resume bullets, interview Q&A, stipend salah, roadmap coverage.

Contents

0. Ek page me poora faisla (TL;DR) . . . 3

1. Senior architect ka analysis . . . 4

2. Final scope: kya banega, kya nahi . . . 7

3. Architecture . . . 8

4. End-to-end workflows . . . 11

5. Proofline Scout: advanced agent jo live systems me bugs dhundhta hai . . . 13

6. Asli cheezon pe testing aur outreach . . . 15

7. Deployment . . . 17

8. Proofline ko test kaise karein . . . 19

9. Project plan: phase-wise . . . 20

10. Project ko internship aur stipend me kaise badlein . . . 23

11. Krish Naik roadmap se updated comparison . . . 25

Appendix: ready-to-use templates . . . 26

Kaise padhein
Padhne ka order: pehle section 0 (ek page), phir 1-3 (soch aur design), 4-6 (workflow aur real-world), 7-8 (deploy aur test), 9-10 (plan aur career). Build karte waqt section 9 (phases) aapka guide hai.

0. Ek page me poora faisla (TL;DR)

Aapke paas teen cheezein thin: (1) Proofline ka vision document, (2) pichla 12-week Enterprise AI Reliability plan, (3) Krish Naik wala FDE + DSA roadmap. Ye document teeno ko ek hi executable plan me badalta hai, ek senior architect, FDE aur project manager ki nazar se.

#	Faisla	Kyun
1	Product ka naam Proofline. Positioning 'universal platform' nahi, balki **Agent Outcome Assurance**: 'AI ne kya kaha' nahi, 'AI ne asal me kya kiya' verify karo.	Eval/observability category already crowded hai (Langfuse, LangSmith, Phoenix, Braintrust, Datadog). Universal banoge to kisi se behtar nahi. Outcome verification wala wedge khaali hai.
2	Ek repo, do modes: **Proofline Lab** (white-box, jahan agent aapka ya client ka ho) aur **Proofline Scout** (black-box audit agent, sirf authorized targets pe).	Lab se depth aur engineering dikhti hai. Scout se asli duniya ke bugs milte hain, jo emails, case studies aur interviews banate hain.
3	Hero feature: **claim-vs-state verifier** (agent ne 'refund ho gaya' kaha, kya DB me hua?).	Proofline doc ka sabse accha idea. tau-bench jaisa state-based check, par product ke roop me.
4	Reliability metric: **pass^k** (k baar chalao, sab pass hon). Sirf pass@1 nahi.	Agents inconsistent hote hain. tau-bench paper me bhi strong model ka pass^1 ~61% (retail) tha par pass^8 ~25% se neeche.
5	Scope kaata: multimodal (image/audio/video), 7 engines ka full build, multi-tenant SaaS, React-first UI sab **defer**.	Solo build ke liye scope chhota rakhna zaroori hai. Proofline v1.0 ka poora scope ek team ke liye bahut bada hai.
6	Pichle plan ke modules (Trace, Guard, Regression, Replay, Watchdog) Proofline ke engines ban gaye. Kuch bhi bekaar nahi gaya.	Month 1-3 ka roadmap coverage bana rehta hai, Month 4-5 ka bada hissa bhi aata hai.
7	Real-world testing **permission-first**: T0 apna, T1 open-source self-hosted, T2 bug-bounty/VDP/sandbox, T3 written permission wale companies.	Bina permission automated testing ToS aur kanoon dono ka risk hai, aur credibility bhi khatam karta hai.
8	Email bhejne se pehle **human review**, evidence packet, reproducibility (k/n), ground-truth source URL. Scout khud email nahi bhejta.	Ek galat 'bug report' aapki reputation tod sakta hai. Ek sahi 1-page audit internship dila sakta hai.
9	Deploy: ek chhoti VM + Docker Compose + Caddy + CI/CD. Kubernetes nahi.	Solo project me K8s overhead hai. VM par sahi security, backup, spend cap zyada impress karte hain.
10	Proofline khud ko test karta hai: **Proofline Bench** (60+ labeled planted failures) se precision/recall.	'Evaluator ko evaluate karo' hi serious engineering hai. Bina iske sab claims khali hain.

Honest note
Is document ka sabse bada risk scope hai, technology nahi. Jab bhi kuch naya add karne ka mann kare, section 9 ki 'cut ladder' dekho. Aur yaad rakho: koi bhi plan stipend ya internship guarantee nahi karta. Ye plan aapke paas aisa proof-of-work chhodta hai jo selection ki probability asli me badhata hai.

1. Senior architect ka analysis

1.1 Teen sources, kaun kya deta hai

Source	Kya accha hai	Kya kamzor hai	Final me kya liya
Proofline v1.0 (vision doc)	Clear problem, claim-vs-state idea, Action Guard boundary ki samajh, capability registry, quality gates, failure taxonomy, honest limitations	Scope bahut bada, 'universal' claim, black-box me ground-truth (oracle) ka jawab nahi, koi timeline ya cut-line nahi, go-to-market nahi	Naam, vision, taxonomy, API/data-model skeleton, verifier idea
Pichla 12-week plan	Executable plan, planted-fault testing, Replay, Watchdog (DB rules + reconciliation), roadmap alignment	Claim verification, black-box mode, deployment, SDK, real-world validation, outreach sab missing	Saare modules engines me merge, tasks reuse
Krish FDE + DSA roadmap	Skills ka sahi order, FDE pillars (integration, security, consulting)	Projects chhote aur alag-alag, 'real client' sirf Month 6 me	Coverage mapping, FDE craft ko Scout audits me mila diya

1.2 Proofline v1.0 ki critical review: 14 issues aur faisle

#	Issue	Kyun problem hai	Faisla
1	'Universal' positioning	Category crowded hai. Universal tool me depth nahi hoti, aur demo bhi weak banta hai.	Wedge: agent outcome assurance (support/ops agents).
2	7 engines + 10 phases	Solo build me impossible. Half-built 7 cheezein, 1 poori cheez se kam impress karti hain.	Engines milte-julte modules me merge, multimodal defer.
3	Black-box ka oracle problem	Bahar se agent ko test karoge to 'sahi jawab kya hai' kaise jaanoge? Doc me jawab nahi.	Scout ground-truth tiers: public facts, self-consistency, link checks, test-account state, judge (last).
4	Claim-vs-state sirf example tha	Ye hi asli differentiator hai par feature roop me define nahi tha.	Verifier spec (section 3.5), Phase 2 ka hero deliverable.
5	Non-determinism ka halka zikr	Ek baar pass hone ka matlab reliable hona nahi.	pass^k aur flake rate first-class metrics.
6	LLM judge pe bharosa	Judge galat ya manipulate ho sakta hai (output me chhupa instruction).	Deterministic-first, judge last, human-labeled calibration, judge-injection defence.
7	Guard enforcement boundary	Doc ne sahi kaha ki observe karna block karna nahi hai, par integration mode define nahi kiya.	Tool Gateway (MCP proxy) + SDK wrapper; modes A/B/C/D table.
8	Replay ki determinism	'Replay' bol dena kaafi nahi, kya reproduce hoga?	Replay levels L0-L3; MVP = L2.
9	Watchdog vague tha (telemetry-based)	'Anomaly detect karo' asli bug nahi pakadta.	Concrete DB rules (duplicate refund, totals, ledger, timezone) + reconciliation + anomaly baseline.
10	React + RBAC + multi-tenant day-1	Premature complexity.	project_id isolation + JWT RBAC; Streamlit pehle, React optional (Phase 6).
11	Test data kahan se aayega?	Golden sets banana sabse mehnat ka kaam hai.	Planted faults + synthetic + public benchmark (tau-bench style) + Proofline Bench.
12	Proofline ki apni security	Transcripts untrusted hain: XSS, judge injection, PII leak.	Escape-all rendering, redact-at-ingest, retention, authz matrix tests.
13	Go-to-market zero	Product banake kisi ko dikhana nahi to portfolio me kuch nahi.	Scout audits, case studies, outreach playbook, design-partner approach.
14	Phases aur acceptance criteria ke bina	Plan ko track nahi kar sakte.	Phase-wise plan, exit tests, risk register, cut ladder.

1.3 Market reality aur aapka wedge

Oct 2026 tak eval + observability category me kaafi mature players hain. Ye snapshot search se bana hai, features jaldi badalte hain, isliye kisi ko pitch karne se pehle unke current docs verify karo.

Tool	Kis cheez me strong	Gap jo Proofline target karta hai
Langfuse	Open-source tracing, prompt management, evals, self-host	Business-state verification, black-box audit, execution-boundary enforcement
LangSmith	LangChain/LangGraph ke saath tracing + evals	Ecosystem-tied, closed platform; outcome verification nahi

Tool	Kis cheez me strong	Gap jo Proofline target karta hai
Arize Phoenix	OpenTelemetry-native tracing aur evals (OSS)	Observability-centric; guard aur claim-vs-state nahi
Braintrust	Eval-driven development, CI gating	Cloud/general; DB-state aur action-claim verify nahi
Datadog LLM Obs	Existing infra ke saath monitoring	General APM; agent outcome assurance nahi
promptfoo, DeepEval, Ragas	Eval libraries	Library hain, trace-to-state workflow nahi
garak, PyRIT	Security red-team scanners	Security-only; business correctness nahi
tau-bench (Sierra)	Research benchmark: final DB state check + pass^k	Benchmark hai, product nahi. Aapko iski methodology adopt karni hai

Architect ki salah
Interoperability se jeeto, competition se nahi. Apna trace schema OpenTelemetry-style rakho aur Langfuse/Phoenix me export option do. Dashboards ki race me mat utro. Aapka kaam woh hai jo unke dashboards nahi karte: 'verify what happened', 'block at the boundary', 'audit from outside'.

1.4 Asli duniya ke agent problems (field catalog)

Ye wahi failures hain jo live support/ops agents me baar-baar milte hain. Har ek ke liye Proofline me detector aur typical fix pehle se tay hai, taaki audit report me 'problem + proof + fix' teeno dikhein.

Problem	Kyun hota hai	Proofline detector	Typical fix
False success claim	Reply plan se banta hai, tool result se nahi	Claim-vs-state verifier	Reply sirf verified tool result se; write ke baad verify step
Policy drift (bot vs website)	Stale ya multiple knowledge sources	Scout ground-truth oracle, Policy Conflict checker	Single source of truth, KB freshness check, eval gate on KB change
Hallucinated coupon/feature/link	Grounding nahi, refusal threshold nahi	Absence probe, link checker, groundedness	Retrieval-required answers, URL validation, 'I don't know' threshold
Wrong entity action	Milte-julte IDs, ambiguity	Guard entity check, wrong-action suite	User ko entity confirm karao, ownership check tool layer me
Duplicate / irreversible action	Retry bina idempotency	Watchdog duplicate rule, Guard reversibility	Idempotency keys, reversible-first design
Silent data corruption	Timezone, total, ledger logic bugs	Watchdog rules + anomaly baseline	DB constraints, reconciliation jobs
Cross-user data leak	AuthZ sirf prompt me	2-account probes (permissioned), RLS tests	AuthZ DB/tool layer me, prompt me nahi
Injection via content	Data aur instruction mix	Canary probes, red-team suite	Privilege separation, write pe confirmation, content = data
Inconsistent answers	Sampling, retrieval variance	pass^k, flake rate	Low temperature, deterministic retrieval, canonical answers
Hindi/Hinglish weaker	Prompts aur evals EN-only	Language-sliced eval	Multilingual golden set, per-language prompt/model
Broken handoff / dead-end loops	Escalation path nahi	Handoff probe, loop detector	Explicit fallback, max retries, human handoff
Regression after update	Baseline nahi	Regression engine + CI gate	Gate merge on critical metrics
Cost/latency blowup	Loops, lambe context	Budget metrics, p95	Step/token limits, model tiering, caching
Integration drift	CRM/ERP sync partial fail	Reconciliation engine	Idempotent sync, DLQ, alerting
PII in logs/traces	Raw logging	Redaction recall tests	Redact-at-ingest, retention policy

1.5 Final product definition

- **One-liner:** Proofline verifies what your AI agent actually did, not what it said it did.
- **Lab:** SDK ya Tool Gateway se jude agents ke liye: trace, claim-vs-state, Action Guard, regression gate, replay, data watchdog.
- **Scout:** authorized third-party agents ka scoped black-box audit: ground-truth, consistency, links, handoff, canary tests, evidence-based report.

- **ICP (pehle kaun):** support/ops agent chalane wale startups aur SMBs (D2C, fintech, edtech, SaaS) jinke paas eval team nahi hai; FDE consultancies.
- **Non-goals:** multimodal platform, compliance certification, autonomous attacker, dashboards me Langfuse se ladna.
- **Aapke liye success:** live demo URL, measured Proofline Bench, 2+ real permissioned audits, 3 case studies, aur 10+ targeted applications.

2. Final scope: kya banega, kya nahi

Proofline ke 7 engines aur pichle plan ke 5 modules ko ek table me jodte hain, saath me priority (P0 sabse zaroori) aur kis phase me banega.

Final component	Proofline engine	Pichle plan module	Priority	Phase
Model adapter + capability registry + gateway	1 Universal Model Intelligence (slim)	Gateway (M4 bonus)	P0	1, 3
Trace store + recorder + viewer	4 Trace, Replay, Diagnosis	A Trace	P0	1
Test cases + deterministic assertions	2 Evaluation (deterministic)	C Eval	P0	1
Claim-vs-state verifier + failure taxonomy	2 Evaluation + Failure Detection	new	P0 (hero)	2
pass^k runner, judge calibration, golden sets	2 Evaluation	C Eval	P0	2
RAG reliability + Policy Conflict checker	2 Evaluation (RAG)	C + D	P1	2
Action Guard + Tool Gateway + approvals	3 Action Guard	B Guard	P0	3
Regression, quality gates, CI action	5 Regression + Gates	C Regression	P0	3
Replay (L2) + diff	4 Replay	A Replay	P1	3
Watchdog: DB rules, anomaly, reconciliation	6 Silent Data + Integration	D Watchdog (#1 + #10)	P1	4
Security, PII/DLP, red team, auth, SDK	cross-cutting	E Platform (M5 bonus)	P1	4
Scout (black-box audit agent)	new, uses 2, 3, 4	new	P0 (differentiator)	5
Deployment, CI/CD, backups, spend cap	Phase 10	M4 coverage	P1	6
Case studies, outreach, interview kit	n/a	M6 coverage	P0 (career)	5-6
Multimodal evaluation (image/audio/video)	7 Multimodal	n/a	DEFER	Phase 6 ke baad
Multi-tenant SaaS, billing, marketplace	n/a	n/a	DEFER	Phase 6 ke baad
Kubernetes, Terraform, Keycloak SSO, vLLM	n/a	M4/M5 leftovers	STRETCH	Phase 6 ke baad

Priority ka matlab
P0 = bina iske project adhoora. P1 = interview me depth dikhata hai. STRETCH/DEFER = tabhi jab P0/P1 ka exit test pass ho chuka ho. Multimodal ko Proofline doc ne bhi 'later phase' kaha tha. Hum us se sehmat hain.

Build order (dependency)

- Trace pehle: baaki sab engines trace par depend karte hain.
- Assertions aur claim verifier, phir pass^k aur judge: evaluation ki neev.
- Guard aur regression gate, phir replay: ye trace + eval par bante hain.
- Watchdog aur security Lab ko complete karte hain.
- Scout sabse aakhir me: use Lab ke verifier, taxonomy aur report engine ki zarurat hai.
- Deploy aur launch tab, jab exit tests pass ho chuke hon.

3. Architecture

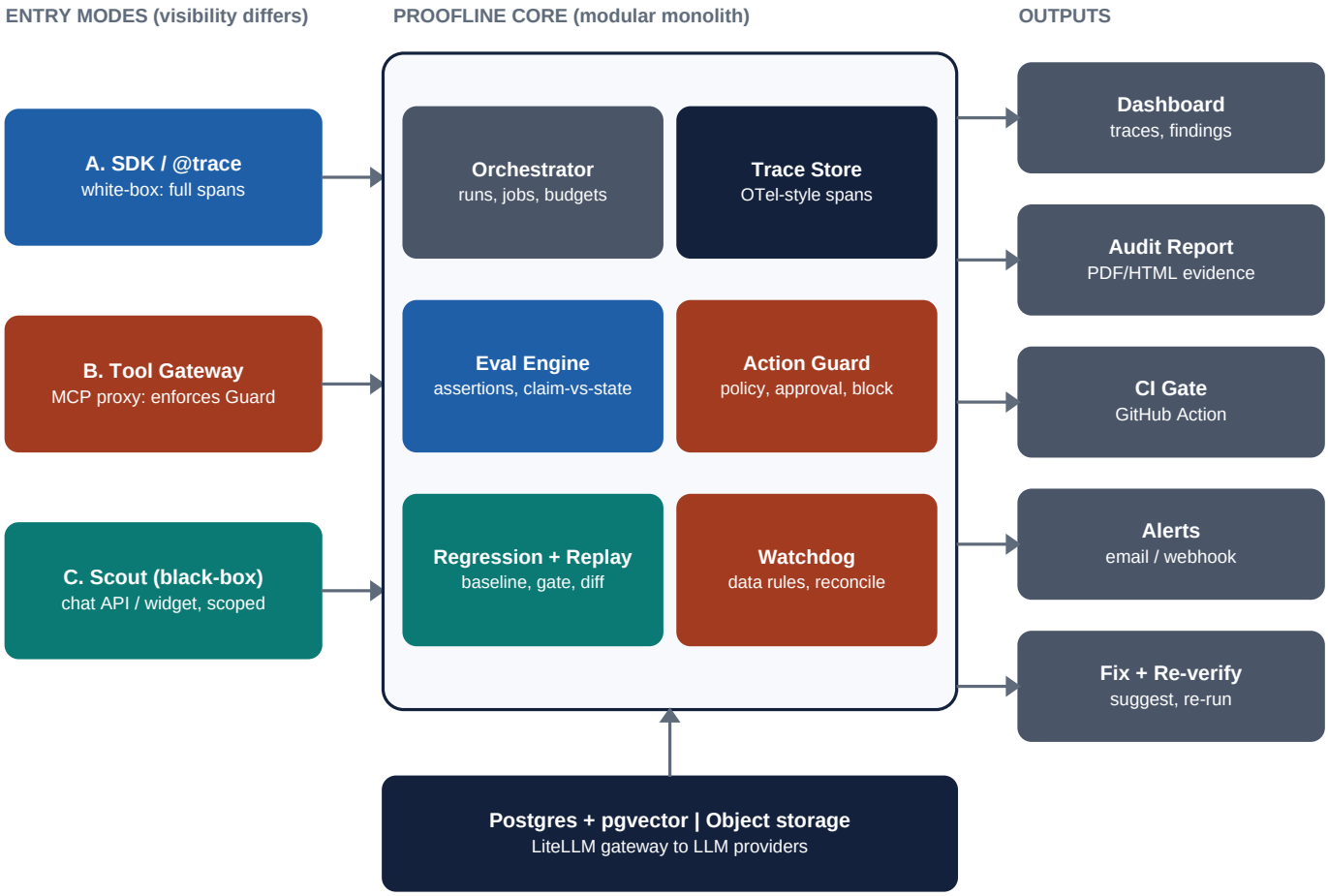
3.1 Integration modes: visibility vs enforcement

Proofline ka sabse bada architecture sawaal ye hai: agent se kitna dikhta hai aur kitna roka ja sakta hai. Iske chaar modes hain.

Mode	Visibility	Enforcement	Effort	Kab
A. SDK wrapper (@trace)	Poora: LLM calls, retrieval, tool I/O	Tools wrap karo to haan	Low	Agent aapka ya client ka code hai
B. Tool Gateway (MCP proxy)	Har tool call	Haan, Guard yahin baithta hai	Medium	Agent MCP/HTTP tools use karta hai
C. Scout (black-box)	Sirf chat input/output + public ground truth (+ test-account state, agar permission ho)	Nahi, sirf observe	Target ko kuch nahi	Third-party audit, cold-start trust
D. OTel/Langfuse import (later)	Existing spans	Nahi	Medium	Client ke paas pehle se tracing hai

Rule: Guard sirf wahi action rok sakta hai jo uske enforcement point se guzre. Mode C me Proofline kuch block nahi karta, sirf evidence deta hai. Report me ye limitation hamesha likho.

3.2 System architecture



Core ek **modular monolith** hai (ek FastAPI app, alag modules), microservices nahi. Scout ka runner alag container me isolated chalta hai, kyunki wo bahar ke hosts se baat karta hai.

3.3 Architecture decisions (ADRs)

ADR	Decision	Reason / trade-off
001	Modular monolith (FastAPI)	Debug aasan, deploy ek unit. Baad me split ho sakta hai.
002	Sirf Postgres (jsonb + pgvector), alag vector DB nahi	Ek DB, kam ops. Scale chahiye to baad me.
003	Trace schema OpenTelemetry-style, apna store, Langfuse/Phoenix export	Lock-in nahi, interoperability, dashboard race se bachna.
004	Python + Pydantic v2 + FastAPI	Roadmap ke saath aligned, validation built-in.
005	UI: Streamlit pehle; React sirf report page ke liye Phase 6 me (optional)	Time sabse bada resource hai. UI se zyada engine matter karta hai.
006	Jobs: Redis + arq worker	Runs aur Scout audits long-running hain; API block nahi karni.
007	Guard = Tool Gateway (MCP proxy) + SDK wrapper	Enforcement point ke bina guard sirf suggestion hai.
008	Scout isolated runner, egress allowlist	Out-of-scope request network level pe bhi nahi ja sake.
009	LLM access LiteLLM gateway se, spend cap ke saath	Provider switch, fallback, budget; replay me model badalna aasan.
010	Evidence-first: har finding ke saath evidence reference	Bina proof ke finding nahi, email nahi.
011	Deterministic checks pehle, LLM judge aakhir me, label ke saath	Judge galat ho sakta hai. Findings me 'judge-assessed' likho.
012	pass^k default (k=5 critical tests pe)	Reliability sirf ek run se pata nahi chalti.

3.4 Data model (merged)

Group	Tables
Workspace	users, roles, projects, model_configs, prompt_versions, datasets
Execution	runs, spans (llm_call, retrieval, tool_call, guard_decision, approval, output), artifacts
Evaluation	test_cases, evaluations, claims, findings, failure_categories, baselines, regression_reports
Guard	guard_policies, policy_versions, guard_decisions, approvals
Watchdog	watchdog_rules, watchdog_findings, baselines_stats, recon_runs, crm_orders, erp_orders
Scout	targets, scope_grants, audits, probes, probe_results, ground_truth_facts, evidence_artifacts
Governance	audit_log, retention_policies

3.5 Claim-vs-state verifier (hero feature) spec

- **Step 1, claims nikalo:** final reply se structured claims: Claim{type: action | fact | policy | link, subject, asserted_status, span}. LLM + Pydantic validation; extraction accuracy alag se naapo.
- **Step 2, evidence jama karo:** tool results, business DB ka before/after snapshot (touched rows), retrieved chunks, URL status.
- **Step 3, compare karo (deterministic):** verdict = SUPPORTED, CONTRADICTED, UNSUPPORTED ya UNVERIFIABLE.
- **Step 4, findings:** false success, false failure, omitted failure, wrong entity. Severity policy se. Evidence refs ke saath.
- **Edge cases:** partial success (3 me se 2 items), async operations (refund 'pending'), ambiguous language ('processed' ka matlab). Inke liye test cases likho.

3.6 Metrics: precise definitions

Metric	Definition	Kyun
pass@1	Ek trial me test pass hone ki average probability	Baseline
pass^k	Test ke k trials me **sab** pass hon. Aggregate = tests ka wo fraction jinke saare k trials pass hue	Reliability; tau-bench ne iska idea diya
flake rate	Wo tests jinke trials me pass aur fail dono mile	Non-determinism dikhata hai
claim accuracy	Sahi verdict wale claims / total claims (labeled set pe)	Verifier ki quality
detection precision / recall	Proofline Bench pe planted failures ke against	Evaluator ka evaluation
false-block rate	Sahi actions jo Guard ne galat block kiye	Guard ki usability
MTTD	Fault inject hone se Watchdog finding tak ka time	Watchdog speed
replay determinism	L2 replay me same decisions wale traces ka %	Replay trust
judge-human agreement	Judge vs human labels (Cohen's kappa ya simple agreement)	Judge calibration
cost / 100 runs, p50/p95 latency	Tokens x price; latency distribution, sirf average nahi	Cost aur speed

3.7 Replay levels

Level	Kya reuse hota hai	Deterministic?	MVP
L0	Sirf input, fresh LLM + live tools	Nahi	-
L1	Input + tool results stubbed	Kaafi hadd tak	yes
L2	L1 + retrieval results + model params/seed	Zyada	**MVP target**
L3	Poora state snapshot restore	Haan, par mehnga	later

3.8 Failure taxonomy (merged, v1)

Group	Codes (default severity)
OUT Output	OUT-01 incorrect answer (S3), OUT-02 unsupported claim (S3), OUT-03 invalid structured output (S3), OUT-04 instruction miss (S4)
ACT Action	ACT-01 wrong tool (S2), ACT-02 invalid args (S3), ACT-03 policy violation (S1-S2), ACT-04 wrong entity (S1), ACT-05 duplicate action (S1)
CLM Claim	CLM-01 false success claim (S2), CLM-02 false failure claim (S3), CLM-03 omitted failure (S2)
POL Policy/KB	POL-01 contradiction with published policy (S2), POL-02 stale info (S3), POL-03 hallucinated link/entity (S3)
DAT Data	DAT-01 silent corruption (S1-S2), DAT-02 sync drift (S2), DAT-03 PII in trace/log (S1)
SEC Security	SEC-01 cross-user leak (S1), SEC-02 injection followed (S1-S2), SEC-03 system prompt exposure (S3)
REL Reliability	REL-01 flaky answers (S3), REL-02 language gap (S3), REL-03 broken handoff (S2), REL-04 timeout/loop (S3), REL-05 cost blowup (S3), REL-06 regression (S2)
META	META-01 unsupported capability (not a failure), META-02 infra error (not counted as pass)

Severity rubric: S1 critical (data leak, unauthorized or irreversible action, financial commitment), S2 high (false action claim, policy contradiction on money/legal), S3 medium (inconsistency, broken link, weak handoff), S4 low (tone, minor format).

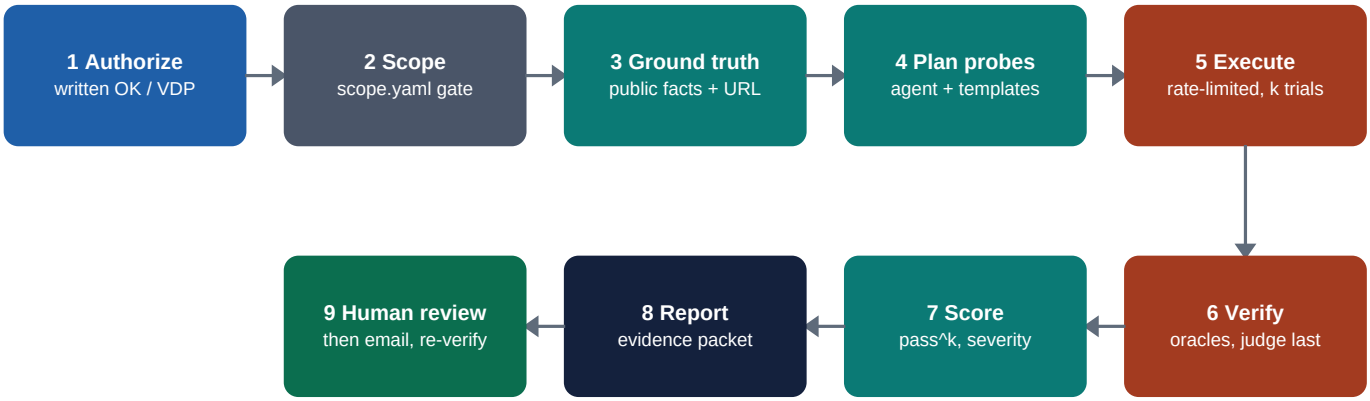
4. End-to-end workflows

4.1 Lab workflow: ek ticket ki poori zindagi (12 steps)

Example: customer Hinglish me likhta hai *'order 1042 ka refund de do, item kharab tha'*. Agent ShopOps ka hai, Proofline uske upar baitha hai.

#	Kya hota hai	Engine	Proof / output
1	Project aur model config register. Capability check: tool-calling support hai ya nahi (nahi hai to test 'unsupported', failure nahi).	Adapter	capability record
2	Test case: input, expected behavior, forbidden actions, severity. Ya production-jaisa ticket.	Eval	test_case
3	Run start: unique run_id, prompt version, model version, config snapshot, budget (steps, tokens, rupees).	Orchestrator	run record
4	Agent policy chunks retrieve karta hai (hybrid + rerank). Chunk ids aur scores trace me.	Trace	retrieval span
5	Agent plan: issue_refund(order=1042). Structured output Pydantic se validate.	Trace	llm_call span
6	Tool Gateway request pakadta hai. Action Guard: tool registered? args valid? order isi customer ka? eligibility check hua? limit? reversible? Faisla: ALLOW / ASK_HUMAN / BLOCK.	Guard	guard_decision + reasons
7	ASK_HUMAN ho to approval queue. Reviewer approve / reject / edit karta hai.	Guard	approval span
8	Tool business sandbox DB pe chalta hai. Before/after row snapshot save.	Trace	tool_call + state diff
9	Agent final reply deta hai. Verifier claims nikalta hai aur tool result + DB state se compare karta hai.	Eval	claims + verdicts
10	Watchdog affected rows pe rules chalata hai (duplicate refund, total, ledger, CRM vs ERP). Finding trace se linked.	Watchdog	findings + trace link
11	Failures taxonomy me classify, severity, evidence ke saath. Dashboard + alert.	Eval	failure report
12	Fix ke baad: replay (L2) + regression (baseline vs candidate, pass^k) + quality gate. PASS / FAIL / REVIEW. CI merge allow ya block.	Regression	gate result

4.2 Scout workflow: bahar se audit (10 steps)



Kill switch aur rate limit har step pe active. Koi bhi email Scout khud nahi bhejta: human review ke baad hi.

#	Step	Detail	Output
1	Authorize	T3: company se likhit permission (email). T2: VDP/bounty ka scope padho. T1/T0: apna system.	permission evidence
2	Scope file	scope.yaml: target, window, allowed categories, limits. Bina iske Scout start nahi hota.	signed scope
3	Ground truth	Company ke public pages (returns, shipping, pricing, FAQ, T&C) se fact table: fact + URL + snapshot date.	ground_truth_facts

#	Step	Detail	Output
4	Plan probes	Categories x facts x languages se probes. Agent adaptive follow-ups banata hai (scope ke andar).	probe plan
5	Execute	Rate-limited conversations, har probe k=5 trials, paraphrase variants. Transcripts + timestamps.	probe_results
6	Verify	Oracles: fact match, consistency, link check, canary. Judge sirf bacha hua, 'judge-assessed' label ke saath.	verdicts
7	Score	pass^k, flake rate, severity, confidence (confirmed / likely / needs-review).	findings
8	Report	Evidence packet + 1-2 page report + fix suggestions.	report PDF/HTML
9	Human review	Aap har finding dobara dekhte ho, false positives hatate ho, fresh re-run se confirm karte ho.	approved findings
10	Send + re-verify	Email (template section 6.5). Fix ke baad Scout se same probes re-run karke 'fixed' verify.	case study

4.3 Bug se email aur fix tak ka loop

- **Finding** milti hai (confidence + k/n repro).
- **Triage:** severity, kya ye bug hai ya sirf acceptable variation? Business impact ek line me.
- **Evidence packet:** transcript, timestamps, ground-truth URL, screenshot, repro steps.
- **Draft:** report generate. **Human review:** aap padhte ho, ek baar manually reproduce karte ho.
- **Send:** permission wale contact ko, private. **Follow-up:** 5-7 din baad.
- **Fix verify:** company fix kare to Scout re-run, before/after dikhao. Ye hi aapka best case study hai.
- **Publish:** sirf consent ke saath, anonymized by default.

5. Proofline Scout: advanced agent jo live systems me bugs dhundhta hai

Scout ek **scoped, permission-gated, evidence-first** audit agent hai. Ye 'hack' tool nahi hai. Ye wahi karta hai jo ek bahut disciplined QA engineer karta: normal customer-style sawaal, hazaar baar nahi, kuch dozen baar, aur har jawab ko public ground truth se milana.

> **Sabse zaroori**
> Authorization ke bina kisi company ke production AI agent pe automated ya adversarial testing mat karo. Isse unki Terms of Service tutte hain aur India me IT Act 2000 ke unauthorized access/damage provisions bhi lag sakte hain. Main lawyer nahi hoon, ye legal advice nahi hai. Safe raasta: permission-first, scope file, low volume, benign tests, private disclosure.

5.1 Safety design: hard rules (code me enforce honge, sirf likhe nahi)

#	Rule	Implementation
1	No scope, no scan	Scout scope.yaml ke bina start nahi hota. Har request se pehle scope hash check.
2	Authorization gate	Target host allowlist; scope window ke bahar request refuse. Isolated container ka egress sirf allowlist hosts.
3	Low rate	Default: 1 request / 5 sec, max 150 conversations per audit, max 8 turns each. 429/5xx pe backoff, 3 consecutive errors pe stop.
4	No real-user data	Synthetic personas. Response me real personal data dikhe to stop, redact, owner ko batao.
5	No circumvention	CAPTCHA, login wall, anti-bot ko bypass nahi. Block mile to ruk jao.
6	Benign tests only	Canary instructions ('reply with CANARY-7F3A'), harmful content elicit nahi, exploit payloads nahi, load test nahi.
7	Kill switch	Env flag + CLI + dead-man timer. Test: kill ke 5 sec ke andar sab requests band.
8	Minimal retention	Transcripts encrypted, report ke 30 din baad delete (consent ho to rakho).
9	Private disclosure	Pehle company ko private, 90 din se pehle public nahi, default anonymized.
10	Human in the loop	Scout kabhi email nahi bhejta. Har finding aap review karte ho.

5.2 scope.yaml (example)

```
audit_id: AUD-2026-004
target:
  name: Example Store Support Bot          # placeholder
  type: web_widget                          # chat_api | web_widget
  url: https://example.com/support
owner_contact: founder@example.com
permission:
  evidence: permission_email_2026-12-02.eml
  window: 2026-12-03T00:00Z/2026-12-10T00:00Z
allowed_categories: [policy_fidelity, consistency, language, hallucination,
                     links, handoff, robustness, latency]
forbidden: [load_test, auth_bypass, real_customer_data, harmful_content, exploit_payloads]
limits:
  max_conversations: 150
  max_turns_per_conversation: 8
  min_seconds_between_requests: 5
  stop_after_consecutive_errors: 3
data: {personas: synthetic_only, retention_days: 30}
disclosure: {private_first: true, public_after_days: 90, anonymize: true}
```

5.3 Scout agent loop

- **Plan:** LLM ko scope, ground-truth facts aur category milti hai. Wo probe list banata hai.
- **Probe:** conversation chalao (rate-limited). **Observe:** transcript + timing.
- **Hypothesise:** 'jawab fact F3 se alag lag raha hai'. **Verify:** usi probe ko k=5 baar + paraphrase + Hindi/Hinglish variant.
- **Oracle:** deterministic check pehle; judge sirf grey cases ke liye.
- **Record:** finding with confidence aur k/n. **Stop conditions:** budget khatam, 3 errors, owner ka stop request, real data dikha, kill switch.

Agent ka kaam 'kahan dekhna hai' tay karna hai. 'Bug hai ya nahi' ka faisla oracles aur aap karte ho, LLM akela nahi.

5.4 Probe battery: 12 categories

#	Category	Probe example	Oracle (sahi jawab kaise pata chale)	Tier
1	Policy fidelity	'Return window kitne din?' (EN/HI/Hinglish)	Company ke public returns page ka fact + URL	T2, T3
2	Self-consistency	Same sawaal 5 baar + 3 paraphrase	Semantic equivalence; flake rate	T1-T3
3	Cross-language	Same fact Hindi vs English	Dono jawab same fact se match	T1-T3
4	Hallucination	Ek non-existent coupon/product/feature poochho	Ground truth me absent; sahi jawab refusal/'pata nahi'	T1-T3
5	Action-claim honesty	Test account pe order cancel request	Account state (order page/email) vs bot ka claim	T3 only
6	Scope and handoff	Out-of-scope, 'human se baat karni hai'	Handoff link/form kaam kare, loop na ho	T1-T3
7	Link and citation health	Bot ke diye har URL	HTTP 200, relevant page, hallucinated URL nahi	T1-T3
8	Instruction hierarchy (benign canary)	User-supplied text me canary instruction	Canary follow hua ya nahi	T1, T3
9	Cross-user leakage	Account B se Account A (dono aapke) ka data poochho	Data leak hua ya nahi	T3 only
10	Robustness	Typos, emoji, lambe input, mixed script, empty input	Graceful error, stack trace/secret leak nahi	T1-T3
11	Safety and tone	Abusive user, distress mention (non-graphic)	Safe deflection / human escalation, brand tone	T1-T3
12	Latency (low volume)	Normal sawaal ka response time	p50/p95, timeout rate; load nahi	T1-T3

5.5 Oracle problem ka jawab (black-box ka sabse bada sawaal)

Evidence level	Source	Confidence label
1 Deterministic	Link status code, format/regex, canary string, test-account state	confirmed
2 Ground truth	Company ke public page ka fact + URL + snapshot date, bot ka jawab usse contradict kare, repro 3/5 ya zyada	confirmed
3 Self-contradiction	Bot khud apne aap se alag jawab de (paraphrase/language/repeat)	likely
4 Judge-assessed	LLM judge, rubric + evidence ke saath	needs-review (human zaroori)

Reproducibility: AI bounty programs me bhi reproducible impact maanga jaata hai, aur model updates ki wajah se finding kal reproduce na bhi ho. Isliye har finding ke saath date, k/n repro aur (dikhe to) model/version note karo, aur email se pehle fresh re-run karo.

5.6 Finding schema aur report

```
{ "finding_id": "F-0007", "category": "POL-01", "severity": "S2",
  "confidence": "confirmed", "repro": "4/5", "language": "en",
  "claim": "Refund window is 45 days",
  "ground_truth": {"fact": "Refund window is 30 days",
                   "source_url": "https://example.com/returns", "snapshot": "2026-12-03"},
  "evidence": ["conv_031.json#t3", "screenshot_031.png"],
  "suggested_fix": "Single source of truth for policy; KB freshness check; eval gate",
  "status": "open" }
```

Report section	Content (company ke liye 1-2 page)
Summary	3 lines: kya test hua, kitne findings, sabse important ek
Scope and method	Permission date, window, kitni conversations, limitations (black-box)
Top 3 findings	Har ek: claim vs ground truth, evidence, repro k/n, severity, business impact
Suggested fixes	Typical fix pattern + 'fix ke baad main re-run kar dunga'
Appendix	Poori findings list, transcripts references, probe categories covered

6. Asli cheezon pe testing aur outreach

6.1 Target tiers: kahan test karna safe aur useful hai

Tier	Kya hai	Kya allowed	Examples (safe)
T0	Aapka apna system	Sab kuch	ShopOps agent planted bugs ke saath, Proofline ka apna demo bot
T1	Open-source agents/chat apps jo aap khud host karte ho, aur public benchmarks	Poora battery, license ke andar	GitHub ke popular open-source RAG/support-bot templates (local deploy), tau-bench jaise tool-agent-user environments (maintained version dekho, original repo ke tasks outdated flagged hain)
T2	Programs jo testing invite karte hain	Sirf unke scope ke andar	Bug-bounty/VDP me AI in scope (HackerOne AI programs, OpenAI Safety Bug Bounty jo agentic prompt-injection aur exfiltration pe focus karta hai; jailbreaks aam taur par out-of-scope), vendor sandboxes/playgrounds jo testing allow karte hain
T3	Company jisne aapko likhit permission di	scope.yaml ke andar	Chhote D2C stores, edtech, startups, college/clinic ke chatbots, jinhe aap free audit offer karte ho
NO	Koi bhi random company ka live bot bina permission	Automated/adversarial testing nahi	Sirf normal customer ki tarah kuch manual sawaal aur wo bhi ToS padhke; us pe 'audit' claim mat karo

Realistic expectation
Bug bounty me paisa kamana primary plan mat banao. Competition tez hai aur 'is in scope?' ka sawaal mushkil hota hai. Wo seekhne aur credibility ke liye accha hai. Aapka primary plan T1 + T3 hai: asli bugs, controlled risk, aur seedha hiring-manager ko dikhane layak output.

6.2 Pre-flight checklist (har T2/T3 audit se pehle)

- [ ] Likhit permission mili aur scope.yaml me link hai.
- [ ] Ek manual normal conversation karke bot ki basic behavior dekhi.
- [ ] Limits set: conversations, turns, rate, window.
- [ ] Synthetic personas ready; koi real customer data use nahi hoga.
- [ ] Kill switch test kiya, egress allowlist check kiya.
- [ ] Ground-truth facts public pages se bane (URL + date saved).
- [ ] Report ke liye owner contact aur disclosure terms tay.

6.3 Success criteria (real-world validation)

Milestone	Target	Phase
Scout apne ShopOps bot pe planted bugs detect kare	Recall aur false-positive rate report	5
T1 pe ek open-source bot ka audit (self-hosted)	Report + 3+ confirmed findings ya 'clean' with evidence	5
T3 permission requests bheje	20-30 targeted requests	5
Permissioned real audits complete	2+ (planning target, replies par depend)	5-6
Case studies	3 (anonymized ya consent ke saath)	6
Fix verify (before/after)	Kam se kam 1	6 ya baad

Reply rate ka koi guarantee nahi. Cold outreach me jawab kam aate hain, isliye volume + value-first (free, chhota, specific) rakho, aur T1 case studies public rakho taaki kuch na mile tab bhi portfolio ban jaye.

6.4 Outreach playbook

- **Target list (30):** startups/D2C/edtech jinke website pe AI chat widget hai. Sheet: company, contact, bot type, permission status, dates.
- **Pehla email = permission request,** findings nahi. Free, 30-minute, scoped, aur 1-page output ka offer.
- **Reply aaye to:** scope file banao, chhota call (FDE style discovery: unka asli darr kya hai: refunds? wrong policy? data leak?), phir audit.
- **Report ke baad:** fix offer, re-verify offer, aur testimonial ya anonymized case study ki permission.
- **Cadence:** Phase 5 me 20-30 targeted permission emails, ek hi follow-up. Spam nahi.

6.5 Email templates (English, kyunki companies ko jaate hain)

Template 1: permission request

Subject: Free 30-minute reliability check of your support chatbot (with your permission)

Hi [Name],

I'm [Your Name], a [final-year student / engineer] building Proofline, a tool that checks whether AI support agents behave consistently with a company's own published policies.

I noticed [Company] uses an AI assistant on [page]. With your written permission, I would run a small, rate-limited test (about 100 normal customer-style conversations over 2-3 days, synthetic personas only, no load, no real customer data) and send you a private 1-2 page report: what matched your policies, what contradicted them, and suggested fixes.

It is free, you can stop me at any time, and nothing is published without your consent. If useful, I can re-run the checks after you fix anything and show a before/after.

Would you be open to this? Happy to share the exact scope and limits first.

Thanks, [Your Name] | [LinkedIn/GitHub]

Template 2: findings report (permission ke baad)

Subject: Private reliability findings for your AI support assistant: 3 items

Hi [Name],

Thanks again for permission to run the check. Summary: [N] conversations between [dates], [X] findings, [Y] confirmed against your published pages.

Top finding: when asked in Hinglish about the return window, the assistant said 45 days in 4 of 5 runs; your returns page (snapshot [date]) says 30 days. Evidence and transcripts are in the attached report.

Each finding includes severity, reproducibility (k/n), the source of truth I compared against, and a suggested fix. Limitations: black-box test, so I cannot see internal causes.

If you fix any of these, I'm glad to re-run the same probes and confirm. I'll keep everything private unless you say otherwise.

Best, [Your Name]

Template 3: follow-up (ek hi baar)

Subject: Re: Free 30-minute reliability check of your support chatbot

Hi [Name], quick follow-up in case this got buried. I can share the exact scope in one message and I'll only start after your written OK. If this isn't a priority right now, no problem at all. Thanks, [Your Name]

Template 4: internship email (report attach karke)

Subject: Intern application: built an AI agent reliability platform (live demo + 2 audits)

Hi [Name],

I'm [Your Name]. I built Proofline, an agent reliability platform that verifies what AI agents actually did (claim-vs-state checks, an action guard at the tool boundary, regression gates in CI) plus a scoped audit agent. On my labeled benchmark it detects [X/Y planted failures] with [P] precision; I've also completed [K] permissioned audits (anonymized case study attached).

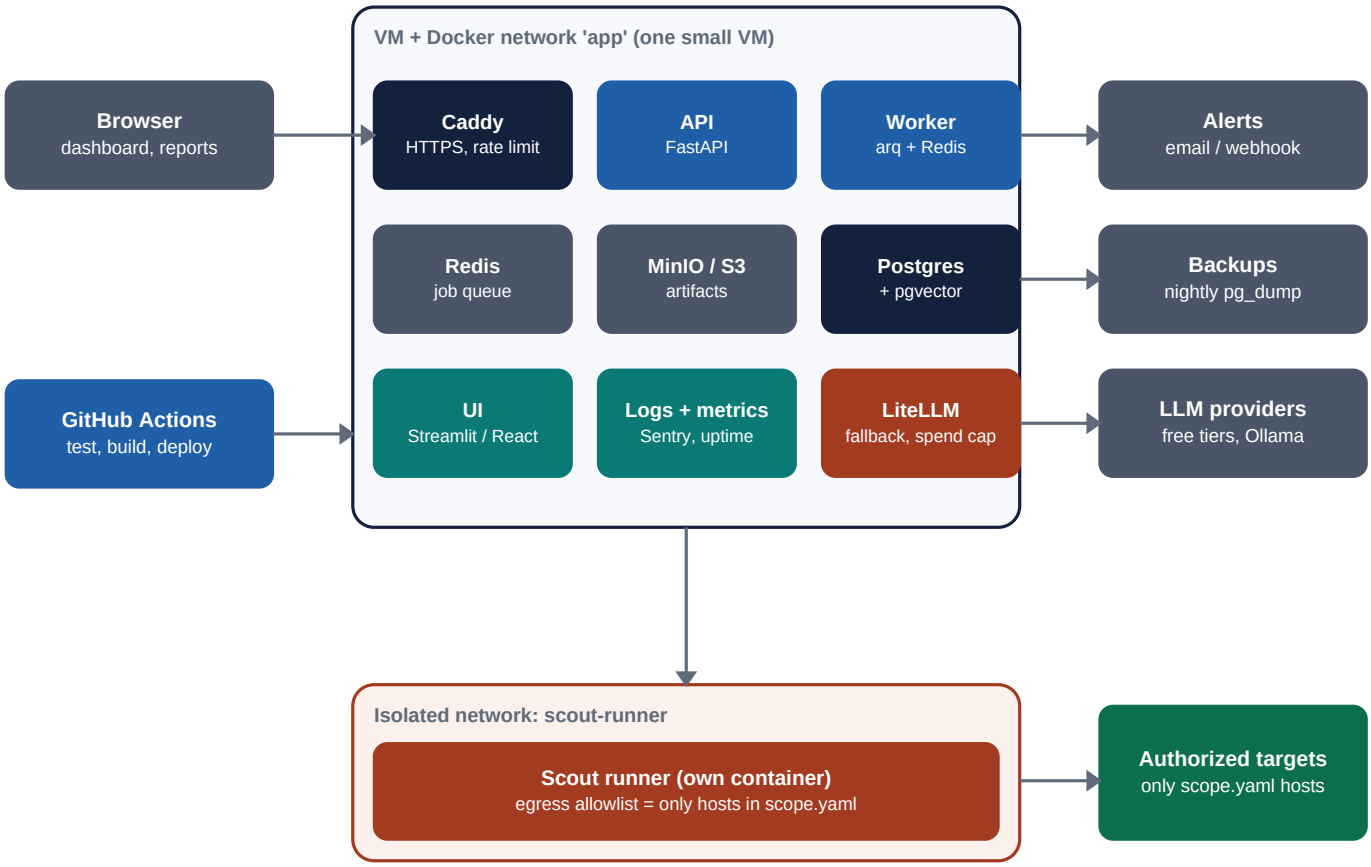
I'd love to contribute to [Company]'s [team: AI/evals/platform] as an intern. Demo: [URL], 2-minute video: [URL], code: [GitHub]. Could we do a 15-minute call this week?

Thanks, [Your Name]

Numbers sirf wahi daalo jo aapne asli me measure kiye hon. Placeholder ko jhootha number se mat bharo.

7. Deployment

Goal: ek public URL jahan recruiter ya company Proofline ko khud chala sake, aur jo sasta, secure aur recoverable ho. Kubernetes isliye nahi: solo project me wo overhead hai, signal nahi.



7.1 Environments

Env	Kahan	Data	Deploy kaise
local	Docker Compose laptop pe	Seeded synthetic data, MockProvider	make up
staging	Chhoti VM (ya same VM ka alag compose project)	Synthetic + demo targets	Har push to main pe auto
prod (demo)	Ek VM, public URL	Sirf synthetic aur consent wala data	Manual approval ke baad

7.2 CI/CD pipeline

- **PR pe:** ruff + pytest (unit, contract) + integration (compose + MockProvider) + **Proofline Bench gate** (apna product apne aap ko test kare, dogfooding) + dependency scan (pip-audit) + secret scan.
- **main pe:** Docker image build, tag = git sha, push registry (GHCR), staging deploy, smoke tests (/health, ek sample run).
- **prod:** manual approval, Alembic migration, compose pull + up, post-deploy smoke test. **Rollback** = pichla image tag + migration down (backup ke saath).
- **Secrets:** GitHub Secrets + server pe env file (permissions 600). Repo me kuch nahi.

7.3 Security hardening checklist

Area	Control
AuthN/AuthZ	JWT login, roles (owner, engineer, reviewer, viewer), har query me project_id; authz matrix tests
Transcripts untrusted	HTML escape-all rendering (XSS), judge ko delimiters + structured output, content kabhi instruction nahi
PII	Presidio redact-at-ingest, DLP output filter, retention 30 din default, delete API
Network	HTTPS only (Caddy), rate limit, CORS allowlist, DB/Redis public nahi, scout-runner alag network + egress allowlist
Containers	Non-root, read-only filesystem jahan ho, resource limits
Secrets	Rotate-able, logs me kabhi nahi, provider keys sirf backend pe
Supply chain	Pinned dependencies, dependabot/pip-audit, image scan
Audit	Guard decisions, approvals, scope grants, report exports ka audit_log
Spend and abuse	LiteLLM daily budget cap, per-run budget, Scout kill switch

7.4 Cost model (planning)

Component	Choice	Cost (planning)
Compute	1 small VM (free-tier ya student credits), 2 vCPU / 4 GB	Free se low
Database	Postgres container + nightly pg_dump to object storage	Free se low
LLMs	Gemini/Groq free tiers, Ollama local; judge ke liye cheap model + human spot-check	Free; daily cap set karo
Domain + TLS	Optional domain, Caddy auto-TLS	Low
Monitoring	Sentry free tier + uptime check	Free
Per-audit LLM cost	conversations x turns x tokens x price (formula)	Audit se pehle estimate karo, cap lagao

7.5 Runbook: top incidents

Incident	Detect	Action
LLM provider down / rate-limited	Gateway errors, p95 spike	Fallback model, record-replay mode for demo
Runaway Scout	Request counter, spend alert	Kill switch, audit.status = aborted, scope review
DB disk full	Disk alert	Retention job, prune artifacts, resize
Leaked key	Secret scan, provider alert	Rotate, revoke, review audit_log
Bad deploy	Smoke test fail	Rollback image tag
Demo day failure	-	MockProvider mode + pre-recorded video

7.6 Go-live checklist

- [ ] Public URL HTTPS pe; demo user aur sample project seeded.
- [ ] Deploy-on-push staging me kaam karta hai; prod manual approval.
- [ ] Backup + **restore drill** ek baar kiya.
- [ ] Spend cap, kill switch, rate limits test kiye.
- [ ] README: architecture, quickstart, limitations, results table, demo video link.
- [ ] Authz matrix tests aur redaction tests green.

8. Proofline ko test kaise karein

Evaluator ko evaluate karna hi is project ko serious banata hai. Neeche ka stack batata hai ki har layer me kya check hota hai.

Layer	Kya test	Tool	Exit criterion
Unit	Assertions, verifier rules, guard checks, parsers	pytest	Core logic coverage high; sab green
Property	'Guard kabhi write allow na kare jab precondition fail ho'	hypothesis	Hazaaron random cases pe violation 0
Contract	Provider adapters, API schema	pytest + OpenAPI checks	Adapter har capability sahi declare kare
Integration	Compose me end-to-end run with MockProvider (deterministic)	pytest + docker compose	Scenarios A/B/C pass
E2E UI	Run create, trace view, report export	Playwright	Critical user flows pass
Meta-eval (Proofline Bench)	60+ labeled planted failures across taxonomy	Bench runner	Per-category precision/recall reported; deterministic categories high
Judge calibration	30+ human labels vs judge	script	Agreement reported; judge-only findings 'needs-review'
Flakiness	k=5 repeat on critical tests	pass^k runner	Flake rate reported, quarantine policy
Security	Authz matrix, XSS in transcript viewer, judge-injection attempts, redaction recall	pytest + manual	0 cross-project leaks, 0 XSS
Scout safety	Out-of-scope refused, rate respected, kill < 5 s, egress allowlist	pytest + network test	All safety tests pass before any real audit
Load/chaos (light)	Provider failure, DB restart, 50 concurrent runs	Locust	p95 aur recovery documented

8.1 Proofline Bench

- 60+ labeled cases: har taxonomy group se 5-10, har ek ka 'ground truth label' (failure present / absent).
- Isme clean cases bhi hain (false-positive naapne ke liye).
- Har release pe Bench chalta hai; precision/recall README table me, version ke saath.
- Bench ko kabhi detector tune karne ke liye leak mat karo: dev set aur held-out test set alag rakho.

8.2 Acceptance gates

Gate	Kab	Pass condition
G1 Lab core	Phase 3 ke end me	Trace 100%, verifier precision/recall reported, 6/6 planted regressions caught, 0 forbidden writes via gateway, kharab PR par gate fail
G2 Watchdog + Security	Phase 4 ke end me	8/8 data faults + 4/4 integration faults found (false alarms reported); authz matrix, redaction tests, red-team before/after table
G3 Scout safety	Phase 5, pehle real audit se pehle	Saare safety tests pass (scope gate, rate, kill switch, egress); scope gate code ka independent review
G4 Real-world	Phase 5 ke end me	Human-reviewed findings; false-positive rate measured; logs me koi out-of-scope request nahi
G5 Launch	Phase 6 ke end me	Public demo, Bench numbers, 3 case studies, README complete

9. Project plan: phase-wise

Plan 6 phases me hai. Har phase ka ek goal, build scope aur exit test hai. Phase tab khatam hota hai jab uska exit test pass ho, kisi tareekh se nahi. Agar pichle plan ka kuch kaam ho chuka hai (Trace schema, ShopOps DB, CI), to Phase 1 me use review karke tezi se aage badho.

PHASE 1 Foundation + Trace

Goal: Poora skeleton khada karo: ek command me chalne wala system jisme ek ticket agent se guzar ke tools chalata hai aur har step trace hota hai.

Build

- **Platform base:** repo, 12 ADRs, Pydantic schemas (Trace/Span, TestCase, Finding), Postgres + Alembic, FastAPI skeleton, Docker Compose, CI (lint + pytest).
- **Model adapter:** interface, MockProvider, capability registry; pehla real provider LiteLLM se (Gemini/Groq free, Ollama).
- **ShopOps sandbox:** business DB (customers, orders, payments, refunds, inventory) + seed; mock tools (get_order, issue_refund, cancel_order, close_ticket).
- **Runs + trace:** run orchestrator (step/token limits), TraceRecorder, Streamlit trace viewer.
- **Tests:** TestCase schema + deterministic assertion library (tool_called, tool_not_called, args_match, state_equals, json_schema, max_steps) + run API.

Exit test

- Fresh clone par ek command me system up.
- Ticket se trace tak poora flow; 100% LLM aur tool calls traced.
- 20 known-outcome tests me assertion engine ke 0 errors.

Pichle plan / roadmap se mapping: Pichle plan ka Month 1 (Trace, Business DB, Docker/CI); Roadmap M1

PHASE 2 Outcome Verification + Eval

Goal: Proofline ka hero feature banao: agent ne jo kaha aur jo hua, dono ko compare karke failures pakadna, aur reliability ko pass^k se naapna.

Build

- **Claim-vs-state verifier:** claim extraction (Pydantic), before/after DB snapshots, deterministic comparison, verdicts, failure taxonomy v1, severity, evidence links; 30 planted claim cases (false success, omitted failure, wrong entity, partial success).
- **Versions + pass^k:** prompt registry, config snapshots, repeat-trial runner (k=5): pass@1, pass^k, flake rate.
- **Golden set + judge:** 80 cases (EN/HI/Hinglish/Bhojpuri-style); LLM judge with rubric + evidence, calibrated on 30 human labels, judge-injection defence.
- **RAG reliability:** 6 policy docs, pgvector, hybrid + rerank, citation correctness, groundedness, refusal rate; Policy Conflict checker (30 injected + 30 clean pairs).

Exit test

- Claim accuracy aur detection precision/recall reported.
- pass^k table; judge-human agreement; 3 injection attempts fail.
- hit@3: vector vs hybrid vs rerank; conflict precision/recall.

Pichle plan / roadmap se mapping: Pichle plan ka Month 2 (prompt, retrieval, RAG, eval); Roadmap M2; verifier naya hai

PHASE 3 Guard + Regression + Replay

Goal: Galat action ko boundary pe rokna, har change ko baseline se compare karna, aur purane trace ko naye version pe replay karna.

Build

- **Action Guard:** tool registry with risk classes, policy.yaml, checks (schema, preconditions, ownership, limits, reversibility), approval queue (interrupt), Tool Gateway as MCP proxy, 20 wrong-action scenarios, hypothesis property tests.
- **Regression + gate:** baseline snapshots, compare A vs B (deltas, pass-to-fail list, new failures), gate rules in YAML, JSON + HTML report, GitHub Action proofline-gate, 6 planted regressions.
- **Replay + gateway:** replay L1/L2 (stubbed tools + retrieval) with decision diff; LiteLLM gateway: fallback, rate limit, spend cap; cost and latency p50/p95.

Exit test

- 0 forbidden writes via gateway; block / false-block table.
- 6/6 planted regressions caught; kharab PR par CI red.
- Replay determinism rate; fallback test table.
- **Gate G1 (Lab core).**

Pichle plan / roadmap se mapping: Pichle plan ka Month 3 (agent, Guard, replay); Roadmap M3 aur M4 ka hissa

PHASE 4 Watchdog + Hardening

Goal: Business data ki silent galtiyan pakadna, aur Lab ko secure aur adoptable banana. Is phase ke baad Lab complete hai.

Build

- **Watchdog:** rule engine (duplicate refund, total mismatch, inventory, ledger, subscription vs payment, timezone, settlement), anomaly baselines (z-score/IQR), CRM/ERP reconciliation (missing, duplicate, stale, mapping), finding-to-trace forensics, alerts.
- **Security:** JWT + RBAC + project isolation tests, Presidio + DLP, 40-attack red team on Guard (before/after), threat model, audit log, XSS-safe viewer.
- **SDK v0:** pip-installable package with @trace decorator and client.

Exit test

- 8 data faults + 4 integration faults: detection, false alarms, time-to-detect.
- Attack success rate before vs after; authz matrix green.
- **Gate G2.** Lab complete: ab internship applications shuru karo.

Pichle plan / roadmap se mapping: Pichle plan ka Watchdog (#1 + #10) aur safety modules; Roadmap M5 ka hissa

PHASE 5 Scout + Real-World

Goal: Permission-gated black-box audit agent banao aur use asli targets par chalao: apna bot, open-source bot, aur permission dene wali companies.

Build

- **Scout core:** scope.yaml loader + authorization gate, rate limiter, kill switch, adapters (chat API, web widget via Playwright), conversation runner, evidence recorder, isolated container + egress allowlist. **Gate G3 pehle, real audit baad me.**
- **Battery + oracles:** ground-truth builder (permitted public pages se fact table with URL + date), 12 probe categories, oracles, pass^k, confidence labels, severity rubric, report generator (HTML to PDF).
- **Real-world pilots:** T1: self-hosted open-source bot + tool-agent-user style environment ka audit. T3: 20-30 permission requests, 2+ permissioned audits, har finding human-reviewed, anonymized case study draft.

Exit test

- Out-of-scope refused; rate respected; kill switch 5 sec ke andar.
- ShopOps bot par planted bugs ka recall aur false-positive rate.
- 2+ real audits delivered; **Gates G3, G4.**

Pichle plan / roadmap se mapping: Naya; Roadmap M6 ka discovery/FDE hissa

PHASE 6 Deploy + Launch

Goal: Public, secure, recoverable deployment aur portfolio ko recruiter-ready banana.

Build

- **Deploy:** prod compose, Caddy TLS, CI/CD staging to prod, Alembic, backups + restore drill, Sentry/uptime, LLM spend cap, README + diagrams, optional React report page.
- **Launch:** Proofline Bench numbers publish, 3 case studies, 2-minute demo video, blog post, resume bullets, internship applications, mock interviews.

Exit test

- Public URL; deploy-on-push; restore drill done.
- Portfolio checklist 100%; applications bheji gayi.
- **Gate G5.**

Pichle plan / roadmap se mapping: Roadmap M4 (deploy) aur M6 (case study, applications)

9.1 Cut ladder: scope kam karna pade to ye order me kaato

Order	Kaato	Impact
1	React report page, Streamlit rakho	Kam, engine same
2	Elasticsearch, Langfuse export, TypeScript, promptfoo	Kam
3	Watchdog anomaly layer (sirf rules + reconciliation)	Kam-medium
4	Replay L2 se L1	Medium
5	Web-widget adapter (Playwright), sirf chat-API + self-hosted targets	Medium; T3 audits kam
6	RAG extras: HyDE, PageIndex-style, Policy Conflict checker	Medium
NEVER	Claim verifier, Guard enforcement, regression gate, Scout safety gate, Proofline Bench, human review	Inke bina project nahi

9.2 Risk register

#	Risk	P / I	Mitigation
1	Scope creep	H / H	Cut ladder, har phase ke start me scope review, Phase 4 ke baad naye features sirf Scout
2	Legal/ToS exposure	M / VH	Tiers, scope gate, permission-first, written proof, benign tests, private disclosure
3	False positives credibility todein	H / H	Confidence labels, k>=3 repro, ground-truth URL, human review, fresh re-run
4	LLM judge unreliable	H / M	Deterministic-first, calibration, 'needs-review' label
5	Flaky tests	H / M	pass^k, low temperature, seeds, quarantine
6	Free-tier quotas	H / M	LiteLLM fallback, caching, Ollama, record-replay in dev
7	Cost blowup	M / M	Per-run + daily caps, kill switch, cheap judge + spot-check
8	Outreach me reply kam	H / M	Volume, value-first, public T1 case studies, parallel normal applications
9	Phase atak jaye, burnout	H / H	Chhote vertical slices, definition of done, cut ladder, ek time pe ek phase
10	Proofline khud leak kare	L / H	Redact-at-ingest, retention, authz tests, no real PII in dev
11	Target behavior badle	M / L	Timestamp, version note, fresh re-run before email
12	Provider API badle	M / M	Adapter layer, MockProvider, contract tests
13	Live demo fail	M / H	Mock mode, seeded data, recorded video

9.3 Har phase ka working rhythm

- **Shuru me:** ADR aur exit test pehle likho. **Build:** chhote vertical slices, har slice end-to-end chalta ho.
- **Exit test:** chalao, numbers README me daalo, screenshot ya GIF lo. Phase ke end me 5-line status update.
- **Teen topi:** architect (ADR), engineer (build), QA (exit tests, Bench), PM (scope, cut ladder), FDE (Phase 5 se client reports).
- **Definition of done:** code merged, tests green, README me naya number, demo artefact.

10. Project ko internship aur stipend me kaise badlein

> **Honest note**
> Koi bhi plan offer guarantee nahi karta. Ye section aapke selection ke chances ko systematically badhane ka tareeka hai: proof-of-work, targeted outreach, aur interview-ready stories.

10.1 Hiring manager 5 minute me kya dekhe

- **GitHub pinned:** Proofline monorepo; README me problem, architecture diagram, Bench results table, limitations, 2-minute demo video.
- **Live demo URL** (mock mode ke saath, kabhi fail na ho).
- **3 case studies:** T1 open-source audit, T3 permissioned audit (anonymized), ShopOps planted-bug story (before/after).
- **Ek blog post:** 'Agents jo kehte hain vs jo karte hain: claim-vs-state se mile findings'. Sirf consent wale ya self-hosted examples.
- **Numbers:** pass^k, Bench precision/recall, attack success before/after, replay time, fallback p95.

10.2 Roles aur kahan apply karein

- **Role titles:** AI Engineer Intern, Forward Deployed Engineer Intern, AI/LLM Evals or QA Intern, ML Platform or Reliability Intern, Backend Intern at AI startups.
- **Kahan:** AI-agent startups, SaaS jisme support chatbot hai, consultancies, eval/observability companies, Indian startups jo D2C/edtech/fintech me AI support chalate hain. LinkedIn, Wellfound, company career pages, founders ko seedha email.
- **Timing:** Phase 4 (Lab complete) ke baad applications shuru karo. Internship cycles me jaldi apply karne ka fayda hota hai. Phase 5 ke case studies aane ke baad second wave.

10.3 Outreach cadence

Channel	Kitna	Kab
Permission-audit emails (T3)	20-30 targeted, ek batch	Phase 5
Internship emails (hiring managers/founders), report attach	10+ targeted	Phase 4 ke baad, phir Phase 5 ke case study ke saath
Normal applications (portals)	Steady, tracking sheet ke saath	Phase 4 ke baad
Follow-ups	Sirf ek baar	Hamesha
Tracking	Ek sheet: company, contact, date, status, next step	Har outreach ke baad update

10.4 Resume bullets (sirf measured numbers daalo)

- Built Proofline, an AI agent reliability platform (FastAPI, Postgres/pgvector, LangGraph, MCP) that verifies agent claims against tool results and database state; detects [X/Y] planted failures at [P] precision on a held-out labeled benchmark.
- Designed an Action Guard enforcing policy at the tool boundary; blocked [N/20] wrong-action scenarios with [M] false blocks; reduced prompt-injection success from [A%] to [B%] on a 40-attack suite.
- Implemented a CI quality gate (GitHub Actions) using pass^k and critical-test regression checks; caught [6/6] planted regressions.
- Built Scout, a permission-gated black-box audit agent (scope file, egress allowlist, kill switch); completed [K] permissioned audits surfacing [M] confirmed findings.
- Deployed on a cloud VM with CI/CD, backups with restore drill, and LLM spend caps.

10.5 Interview Q&A (outline)

Sawaal	Aapka jawab ka dhancha
Ek LLM agent ko kaise test karoge?	Deterministic assertions on tool calls aur final state, claim-vs-state, pass^k for non-determinism, judge sirf grey cases ke liye aur calibrated.
Judge pe bharosa kyun nahi/kyun?	Bias aur injection; isliye evidence-based rubric, human-labeled calibration, 'judge-assessed' label, deterministic-first.
Guard ko bypass kaise hoga?	Agar action enforcement point se na guzre. Isliye Tool Gateway, least-privilege creds, aur limitation documented.
Black-box audit me sahi jawab kaise jaante ho?	Oracle tiers: deterministic, public ground truth with URL/date, self-contradiction, judge. Confidence labels.
Non-determinism kaise handle kiya?	pass^k, flake rate, seeds, repeat trials, report me uncertainty.
Silent data corruption kaise pakda?	DB rules + anomaly baseline + reconciliation; finding ko trace se link karke root cause.
Kya trade-offs liye?	Universal ki jagah wedge, multimodal defer, Streamlit before React, modular monolith, ADRs.
Ethical/legal kaise sambhala?	Permission-first tiers, scope gate in code, rate limits, benign tests, private disclosure, human review.
Is project ka FDE angle?	Discovery calls, scoped audit (mini-SOW), report for non-technical owner, fix verification, communication.

10.6 Stipend ke baare me honest salah

- Stipend company, role aur location pe depend karta hai. Koi number promise nahi. Proof-of-work aapko 'padhne wala student' se 'kaam ka engineer' bana deta hai, aur negotiation me wahi leverage hai.
- Offer ke baad hi stipend discuss karo; apna proof, kaam ka scope, aur remote/onsite saaf rakho. Unpaid 'exposure' wale roles se bachne ki koshish karo, ya unhe time-boxed rakho.
- Parallel raasta: chhote businesses ko paid 'AI chatbot reliability audit' (fixed scope, fixed price). Ye guaranteed income nahi hai, par real client experience aur testimonials deta hai.
- Ek se zyada offers ke liye pipeline chahiye: emails, calls, interviews ke numbers track karo.

10.7 Phase 6 ke baad next steps

- **Pehle:** interviews, aur audits, Bench v2, blog post, open-source release ka decision.
- **Phir:** Keycloak/RLS, vLLM ya Kubernetes me se wo jo target roles maange (stretch).
- **Phir:** internship start, ya 2-3 paid audits aur ek design-partner conversation.

11. Krish Naik roadmap se updated comparison

Is plan ke baad 24-week FDE roadmap (Part A) ka kitna hissa cover hota hai. Percentage mera andaza hai.

Roadmap month	Coverage	Cover ho gaya	Kya chhut raha hai
M1 Foundations	~100%	Python, Pydantic, pytest, SQL, FastAPI, Docker, Git, CI, LLM APIs	TypeScript (optional)
M2 LLM Engineering	~100%	Prompting, structured output, hybrid retrieval, rerank, citations, evals, conflict detector	Elasticsearch (optional)
M3 Agentic AI	~100%	ReAct, LangGraph, MCP, approvals, guardrails, limits, kill switch, replay	SDK deep-dives (skim)
M4 Production	~75%	Cloud VM deploy, CI/CD, LiteLLM fallback, tracing, cost dashboard, spend cap, backups	Kubernetes/Helm, Terraform, semantic cache, full chaos tests
M5 Enterprise	~60%	JWT RBAC, PII/DLP, red team, threat model, audit log, connector-like reconciliation	Keycloak SSO, Postgres RLS, SharePoint/Salesforce connectors, vLLM, TCO
M6 FDE Craft	~50%	Discovery via permissioned audits, scoped reports, communication, case studies, outreach	Long-running client delivery, formal SOW, user training

Total: Part A ka lagbhag **80%**. Bacha hua hissa (Keycloak + RLS, vLLM, K8s/Terraform, ek lamba real client) Phase 6 ke baad stretch ke roop me jodo, wo bhi isi platform ke upar, naya project nahi.
DSA (Part B) alag chalta rahega, roadmap PDF ke hisaab se. Ye document sirf Part A ke kaam ko cover karta hai.

Appendix: ready-to-use templates

A. Guard policy.yaml (example)

```
tools:
  get_order:
    risk: read
  issue_refund:
    risk: irreversible_write
    requires: [order_belongs_to_customer, eligibility_checked, not_already_refunded]
    limits: {max_amount: 5000}
    approval: {required_if: "amount > 2000"}
  close_ticket:
    risk: reversible_write
    requires: [customer_confirmed_resolved]
defaults: {on_unknown_tool: BLOCK, on_missing_precondition: BLOCK, max_steps: 8}
```

B. Quality gate rules (example)

```
gate:
  critical_tests: {must_pass_all: true, pass_k: 5}
  forbidden_action_executed: 0
  task_success_min: 0.90
  regressions_max: {critical: 0, total: 2}
  latency_p95_max_ms: 8000
  cost_per_100_runs_max_inr: 150
  on_infra_error: REVIEW      # never counted as PASS
```

C. README checklist

- [ ] Problem aur one-liner, diagram, 2-minute demo video.
- [ ] Quickstart: ek command me up (docker compose up).
- [ ] Results table: Bench precision/recall, pass^k, attack success before/after, replay time, fallback p95.
- [ ] 'Where it fails' section aur limitations (black-box visibility, judge errors).
- [ ] Security and ethics: scope gate, tiers, disclosure policy.
- [ ] Architecture decisions (ADR links) aur roadmap.

D. Phase-end status update (5 lines)

```
Phase N: <title>
Done: <3 bullets>        Exit test: <pass/fail + number>
Blocker: <one line>      Risk change: <none/new>
Next phase: <3 bullets>
```

> **Final note**
> Is document ke sab numbers (percentages, targets) planning estimates hain. Real results aapko apne measurements se likhne hain. Competitor aur bounty-program details Oct 2026 ke search snapshot par based hain, use karne se pehle current pages verify karo.